

FJ1 ASSIGNMENT

Subject: Robot Programming (21EIC553J)

Title: Comparative Design and Deployment
Strategy for SLAM-Based Autonomous
Navigation in Multi-Floor Indoor
Environments

Submitted by:

Bharathwaj Manoharan
(RA2512053010003)

1. Introduction And Background:

Simultaneous Localization and Mapping (SLAM) is the foundational technology that allows a mobile robot to build a map of an unknown environment while simultaneously estimating its own pose within that map. In this process, the robot incrementally fuses sensor measurements to estimate its motion and to annotate a map with observed obstacles or landmarks. Localization refers to the problem of determining the robot's pose (position and orientation) in a known map or relative to a reference frame. Path planning is the task of computing a collision-free trajectory from the robot's current location to a goal location, typically using the map and knowledge of obstacles. A complete navigation system integrates SLAM for map-building and localization, plus planning algorithms (global and local planners) to produce motion commands.

Indoor multi-floor navigation introduces additional challenges beyond single-floor operation. Academic buildings often have long, repetitive corridors without distinctive features, which can confuse scan-matching and visual odometry. Glass walls and mirrors are common in modern buildings; these reflective surfaces cause LiDAR beams to pass through or reflect unpredictably, resulting in missing or spurious range measurements and inconsistent maps. Dynamic human traffic means moving obstacles appear and disappear; a static map cannot represent them, and moving people can degrade SLAM accuracy if not filtered out. Furthermore, intermittent Wi-Fi or communication outages in an indoor environment mean that the robot cannot rely on cloud services or remote state updates; the navigation system must be robustly self-contained. Finally, limited onboard computation constrains algorithm choices: mapping and planning must be efficient to run in real time.



Figure: Autonomous indoor mobile robot navigating a multi-floor academic building with corridors, elevators, and students walking around.

Key challenges in multi-floor navigation include handling vertical transitions between floors (via stairs or elevators), dealing with aliasing from nearly-identical floor layouts, and maintaining consistency across floors. For example, in an elevator the robot's LiDAR returns do not change while ascending, so the SLAM system must detect altitude change through other means (e.g. a barometric sensor or IMU) and create a separate layer or segment of the map. Repetitive floor plans can cause loop-closure failures (the robot may think it revisited the same place when it's actually on a different floor), so specialized strategies like floor-specific constraints or multi-layer maps are needed. These factors make robust, multi-floor SLAM an advanced problem requiring a careful selection of sensors, algorithms, and system design.

2. Literature Review:

SLAM Algorithms: There exist many SLAM algorithms tailored to different sensors and requirements:

- **Gmapping (RBPF SLAM):** A common 2D LiDAR SLAM (used in ROS) based on the Rao-Blackwellized Particle Filter (RBPF). It incrementally constructs a 2D occupancy grid map while maintaining a particle filter over robot poses. Gmapping is lightweight and works well in moderate-sized indoor spaces, but it can suffer from drift in long featureless corridors and may struggle with loop closure in large maps. It expects planar motion and assumes floor-flat structure, making it suitable for single-floor mapping.
- **Google Cartographer:** A real-time SLAM library that supports both 2D and 3D mapping. Cartographer builds submaps (locally consistent occupancy grids) and maintains a globally-consistent pose graph with optimized constraints. It performs continuous scan matching (ICP or Ceres-based) and uses sparse pose graph optimization for loop closure. Cartographer tends to be more robust to drift than simple filter SLAM, and it has mechanisms to handle 2D LiDAR drift via scan-to-submap matching. However, it requires more CPU and memory. For multi-floor scenarios, Cartographer can be extended to 3D mapping (e.g. using a 3D LiDAR) or run separate 2D instances per floor, but careful management of floor transitions is needed.
- **ORB-SLAM (Visual SLAM):** A feature-based visual SLAM system that works with monocular, stereo, or RGB-D cameras. It extracts ORB features from images, tracks them over time, and creates a sparse 3D map of landmarks. ORB-SLAM performs loop closure detection via

place recognition and maintains a pose graph for global consistency. It offers high accuracy in visually rich scenes and builds a metric map (with scale if stereo or RGB-D). In indoor buildings, visual SLAM can complement LiDAR SLAM: cameras can detect doors or unique visuals in corridors, but they may fail in dim lighting or on textureless walls (e.g. blank corridors). Moreover, glass or reflective surfaces can cause false visual features. Visual SLAM also tends to be more computationally intensive and may require hardware acceleration for real-time performance on limited CPUs.

Other notable SLAM systems include: - **Hector SLAM: A 2D LiDAR SLAM using scan matching (ICP/NDT)** without explicit loop closure (no global optimization). Very fast on CPU, but drift accumulates over time. - **LIO-SAM: A LiDAR-Inertial Odometry and Mapping system** that fuses 3D LiDAR scans with IMU data in a factor-graph approach. High accuracy for 3D mapping, used in multi-level scenarios (e.g. airplanes or warehouse with height). - **RTAB-Map: A graph-based SLAM** that can fuse LiDAR, visual, or RGB-D streams, often used for building 2D/3D maps with loop closure detection.

Localization Techniques: Once a map is available, the robot needs to localize itself within it:

- **Adaptive Monte Carlo Localization (AMCL):** A particle filter approach (commonly used in ROS) that tracks the robot's pose within a known static 2D map. AMCL maintains a set of weighted pose hypotheses and updates them using motion (odometry) and sensor models. AMCL can recover from kidnapped robot situations and handles multi-modal uncertainty. It is robust but can be computationally heavy if many particles are needed, although it can adapt the number of particles.

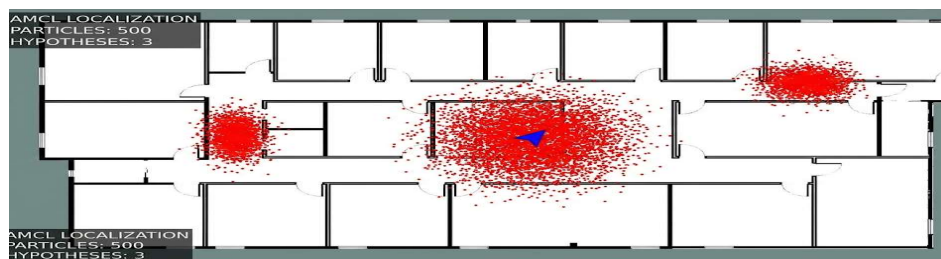


Figure: Particle cloud from AMCL localization in an indoor grid map showing multiple hypotheses

- **Extended Kalman Filter (EKF):** A classical Bayesian filter that fuses odometry and sensor measurements under Gaussian assumptions. In localization, an EKF might fuse wheel odometry and IMU (and possibly ranged measurements to known landmarks or beacons) to maintain a single Gaussian belief over pose. The EKF is lightweight but may fail if the posterior becomes non-Gaussian (e.g. in highly symmetric spaces) and cannot easily represent multiple hypotheses. In practice, the `robot_localization` ROS package (EKF/UKF) is used to fuse multiple odometry sources but assumes some known map or beacons for correction. EKF is often used in state estimation for odometry rather than pure map-based localization.

Other filters include Unscented Kalman Filters (UKF), particle filters (AMCL is one), and specialized approaches like Markov Localization (discrete filter) or graph-based relocalization (matching sensor data to map to get pose).

Navigation Strategies: Planning a path in a known map or map-in-progress involves both global planning and local reactive planning:

- **Global Planning:** Given a static occupancy grid, algorithms like A* or Dijkstra's algorithm compute an optimal path from start to goal. In ROS, `global_planner` (based on A*) or `navfn` (Dijkstra-based) are common. These compute a sequence of waypoints far ahead, assuming the map is accurate. In a multi-floor building, the global planner must also consider connectivity between floors (e.g. detect elevator or stair positions and include them as graph edges). For very large maps or semantically segmented floors, the planner might operate on a topological graph (rooms connected by corridors and stairs) to reduce complexity.
- **Local Planning (Dynamic Window Approach, DWA):** A local planner generates real-time velocity commands to follow the global path while reacting to immediate obstacles (including moving ones). The Dynamic Window Approach (DWA) samples velocity commands (forward speed and rotation) in the control space and simulates short trajectories, scoring them by progress toward the goal and obstacle clearance. The best-scored command is sent to the robot. DWA handles differential-drive kinematics and dynamic limits. Other local planners include Timed-Elastic Band (TEB), which optimizes a deformable trajectory under dynamic constraints, and Pure Pursuit. Local planning uses a local costmap (representing known static obstacles plus dynamic

obstacles perceived by sensors) to avoid collisions.

- **Recovery Behaviors:** In ROS, if the local planner cannot find a valid command (e.g. due to dead ends or odometry drift), recovery behaviors like “clear costmap” or “rotate in place” can be triggered to allow the robot to extricate itself.

3. Conceptual Framework:

The SLAM system can be understood via two main components: a front-end that processes raw sensor data to build local constraints, and a back-end that optimizes the global map/trajectory.

- **Front-End (Sensor Processing):** In each robot time-step, raw sensor measurements are processed. For LiDAR-based SLAM, front-end processing usually means scan matching or registration: matching a new laser scan to the existing local map or previous scan (e.g. using Iterative Closest Point (ICP) or variants). For vision-based SLAM, the front end involves feature extraction (detecting point features like ORB or SIFT in images) and feature matching between consecutive frames. Odometry or IMU can also provide motion estimates. The front end produces motion constraints: e.g. a relative transform between poses, or landmark observations. Data association (matching features or scan points to the map) is crucial.
- **Pose Estimation:** The robot’s pose is estimated by fusing these constraints. In filter-based SLAM (e.g. EKF SLAM or particle filter SLAM), a recursive Bayesian filter is used. In graph-based SLAM, poses at each time step become nodes in a graph, connected by relative transform constraints. For example, an Extended Kalman Filter predicts a pose via motion and updates it with sensor measurements; in a pose-graph, an optimizer later adjusts all poses to minimize error. The fundamental Bayesian update for localization is:

$$bel(x_t) \propto p(z_t | x_t) \int p(x_t | x_{t-1}, u_t) bel(x_{t-1}) dx_{t-1}$$

where x_t is pose, u_t the control/odometry, and z_t the sensor measurement. This underlies particle filters like AMCL.

- **Back-End (Optimization / Mapping):** As the robot moves, a sequence of poses and map constraints is built. The back-end periodically performs optimization to reduce accumulated error (drift). In graph-based SLAM, each pose or keyframe is a vertex in a graph, with edges encoding relative motion or loop closure constraints. The objective is to find the configuration of poses $X=\{x_i\}$ that minimizes the sum of squared errors:

$$X^* = \arg \min_X \sum_{(i,j) \in C} \| z_{ij} \ominus (x_i^{-1} x_j) \|_{\Sigma_{ij}}^2$$

where z_{ij} is the measured relative transform between poses i, j , Σ_{ij} the information matrix, and $\ominus$ denotes pose difference. Optimization methods like Gauss-Newton or Levenberg–Marquardt are used.

- **Loop Closure:** A critical part of SLAM is detecting when the robot revisits a previously mapped area. The front end may attempt to recognize a place (by appearance of scenes or matching scan segments) and propose a loop closure constraint. This constraint, added to the graph, greatly reduces drift by forcing consistency. Loop closure can be challenging in repetitive indoor corridors, because many places look similar. Techniques include visual bag-of-words recognition, scan context (LiDAR global descriptors), or exploiting building semantics. Robust loop closure is vital for multi-floor consistency: floor-specific loop detection can prevent incorrectly matching different floors. For instance, a strategy is to maintain separate “floor labels” in the map and only search for loops within the same floor layer to avoid false closures.

SLAM Pipeline (Simultaneous Localization and Mapping)

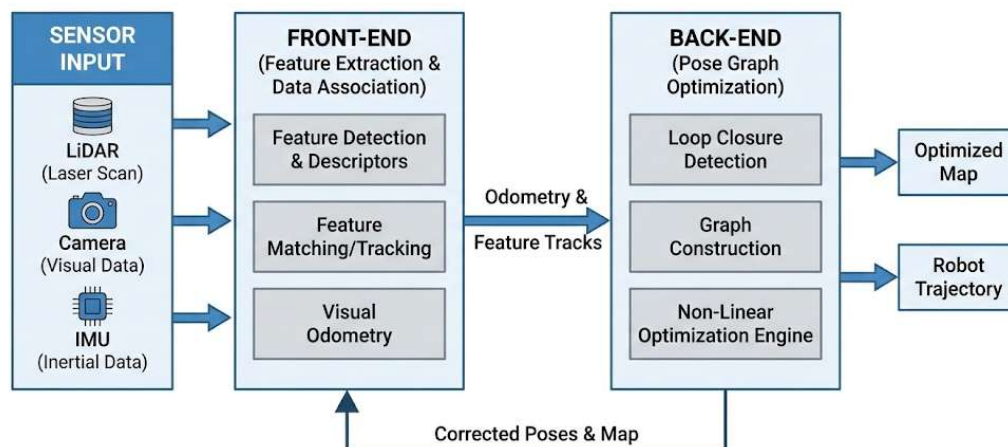


Figure: SLAM pipeline block diagram showing sensor input, front-end feature extraction, and back-end pose graph optimization.

- **Map Representation:** Common map representations include:
 - **2D Occupancy Grid:** A discretized grid where each cell stores the probability of being occupied. Widely used for planar navigation with 2D LiDAR. In multi-floor contexts, one can use one grid per floor or a multi-layered grid (each floor as a separate layer with independent occupancy).

- **3D Point Cloud / Voxel Map:** Used with 3D LiDAR or stereo/RGB-D cameras to build 3D maps (e.g. OctoMap). More expressive but computationally heavier. Multi-floor can be naturally represented in 3D.
- **Feature Map:** A sparse set of landmarks (points, lines). Common in visual SLAM.
- **Pose Graph / Topological Map:** Key poses connected by edges, often used for high-level planning or saving memory.
- **Localization Filtering:** For real-time pose estimation, filters are used. An EKF might fuse odometry and IMU to maintain a pose estimate, correcting it with occasional absolute measurements (like visual re-localization). A Particle Filter like AMCL uses many hypotheses and weights them by how well the laser scan matches the map. AMCL naturally handles multi-modal hypotheses (useful if the map is symmetric). For systems with limited CPU, filter size and sensor rates must be tuned.
- **Sensor Fusion:** To handle challenges (stairs, elevators), multi-modal fusion is important. For example, integrating a barometric altimeter or z-velocity (e.g. UAV altitude) can detect floor changes. A multi-floor SLAM system might use 2D scans for planar mapping and an IMU to track vertical motion, creating a new “map layer” when an altitude threshold is crossed. Floor transitions can also be flagged by event sensors (e.g. elevator button presses, stair detection by tilt).

4. Detailed Analysis:

Comparative SLAM Systems

- **Gmapping:** In a multi-floor academic building, Gmapping’s 2D LiDAR SLAM will build an occupancy grid as the robot moves. It handles planar motion and can create separate maps if the robot does each floor independently. However, Gmapping has limited loop closure ability. In repetitive corridors, Gmapping’s scan matching can be ambiguous and drift can accumulate significantly, potentially stretching corridors in the map. With glass surfaces, Gmapping will see missing walls (lasers ignore glass), leading to incomplete maps; false free space might appear behind glass. Gmapping is lightweight, so it suits a CPU-limited platform, but reliability suffers in symmetric or reflective environments. It also cannot handle vertical transitions on its own; typically, one would stop mapping before an elevator, change the floor map (or use a new map), and rely on external cues for floor changes.

- **Cartographer:** Offers improved robustness in this scenario. Its real-time scan matching and submap scheme allow more stable local mapping. Loop closures in Cartographer can correct drift over long distances, which helps in featureless corridors if loop detection is triggered correctly. Cartographer can manage multiple trajectories (e.g. a new trajectory when changing floors) and then merge them via the pose graph. For glass walls, Cartographer still uses LiDAR, so similar limitations apply (lack of returns), but its robust optimization may partially mitigate spurious scans. Cartographer can run in a resource-constrained CPU if tuned (e.g. reducing submap size or optimization frequency). In terms of multi-floor mapping, Cartographer does not natively support 2D multi-layer maps, but one can maintain a separate instance or use the 3D mode (if a 3D LiDAR is available). Its CPU load is higher than Gmapping, but it provides better accuracy when loop closures are frequent.
- **ORB-SLAM:** Using cameras (mono or stereo), ORB-SLAM provides a dense set of feature landmarks. In indoor corridors, ORB-SLAM can leverage visual textures (like posters, door frames) that LiDAR might not. It inherently performs loop closure through visual place recognition, which could correct drift if the robot sees the same scenes again. On the other hand, ORB-SLAM is sensitive to lighting changes (e.g. hallway lights, shadows) and dynamic objects (people moving), which may cause feature tracking loss or false loop detections. Glass walls can confuse vision (reflections) but might also provide features (the edges of door frames near glass). ORB-SLAM3 (with IMU) is more robust: the inertial data aids through dark or featureless areas. ORB-SLAM requires significant computation, so on limited hardware it might run slower or at lower frame rates. ORB-SLAM also does not directly produce a 2D occupancy map; it yields a sparse 3D map and pose graph. For navigation, one would typically convert its output into a metric map (or use a hybrid approach: visualize camera pose on an existing LiDAR map).
- **Comparison:** In summary, Gmapping is simplest and fastest but more prone to drift; Cartographer is heavier but more accurate with strong loop closing; ORB-SLAM adds rich features but introduces camera-specific failure modes. A practical system in this scenario might combine LiDAR and vision: e.g. run Cartographer for 2D mapping and use a monocular camera SLAM as a relocalization aid, or use LiDAR SLAM for the base map and ORB-SLAM for verifying loop closures in visually distinctive hallways.

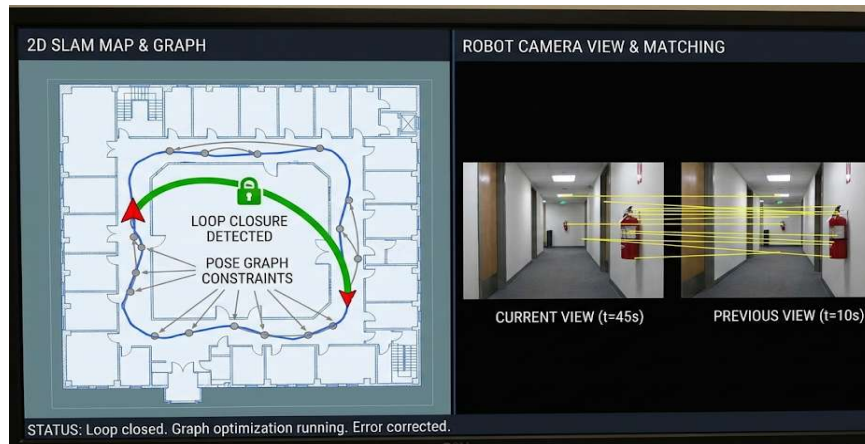


Figure: Visualization of loop closure in SLAM where robot revisits a place and graph constraints are added.

Localization Methods

- **AMCL (Particle Filter):** With a built map (e.g. from Gmapping/Cartographer), AMCL can localize the robot. In multi-floor use, AMCL typically operates on one 2D floor map at a time. If the robot transitions to a new floor, the system would switch the map and reset AMCL's particles to a corresponding region (e.g. elevator exit). AMCL is robust to momentary localization loss if the map has distinguishing features. However, in long uniform corridors, AMCL may distribute particles along the corridor length (multi-modal hypothesis). It recovers by matching scan features (corners, door frames) in the map. AMCL is relatively computationally cheap for a moderate number of particles, so it fits on limited CPU.
- **EKF-based Localization:** A 2D EKF could fuse odometry, IMU yaw rate, and range-bearing measurements to known landmarks (if present). In a map, these "landmarks" could be corners or beacons. Without explicit beacons, an EKF can only fuse odometry+IMU (dead reckoning), which drifts unbounded. Sometimes an EKF is used to fuse multiple odometry sources (wheel + IMU) but not for absolute localization on a map. Compared to AMCL, a pure EKF localization is less reliable when the environment is symmetric (since it maintains a single hypothesis). However, EKF is light on CPU. One use in ROS is the robot_localization package, which can combine multiple sensors (including occasional GPS or visual pose fixes) with an EKF or UKF. For an indoor building, an EKF might fuse LiDAR scan-matching output (as a relative motion measurement) with IMU to produce smoother pose estimates.

- **Comparison:** AMCL is more suited when a detailed map is available and the environment allows scan matching. EKF is more suited for fusing continuous sensor streams (IMU, wheel odom) and requires supplemental absolute updates. In practice, a hybrid approach is common: use AMCL for map-based localization and also run an EKF to smooth the velocity commands or integrate IMU for short-term pose prediction (since AMCL particle updates can be sparse at low scan rate). This combines the strengths of both filters.

Hybrid ROS Architecture

A recommended ROS-based architecture for multi-floor indoor navigation would integrate the mapping and localization modules with both global and local planners. A typical layered architecture is:

- **Sensor Layer:** Includes 2D LiDAR, IMU, wheel odometry, and possibly cameras. LiDAR provides 2D range scans; IMU provides orientation and vertical motion; odometry gives incremental motion; cameras (if used) provide images for a visual SLAM or localization aid.
- **Mapping SLAM Node:** Runs one of the SLAM algorithms (e.g. Cartographer or Gmapping). This node consumes the sensor data and produces a map (global occupancy grid) and real-time odometry/pose estimate. If multi-floor, it could maintain a multi-layered map (multiple 2D layers, one per floor), or switch SLAM contexts when changing floors. Some systems create a new submap for each floor. The SLAM node should periodically save the map (e.g. via map_server) so localization can use it.
- **Localization Node (AMCL):** Once the map is built or loaded, AMCL runs using the SLAM-generated map. It subscribes to scan and odometry, outputting the robot's pose estimate on the /tf transform between map and odom frames.
- **Map Server:** A ROS node that provides the static map to planners. In multi-floor operation, the system could maintain separate map servers (one per floor) and switch which map is active based on detected floor. Alternatively, a single 3D map service or a smart map that encodes floor membership could be used.

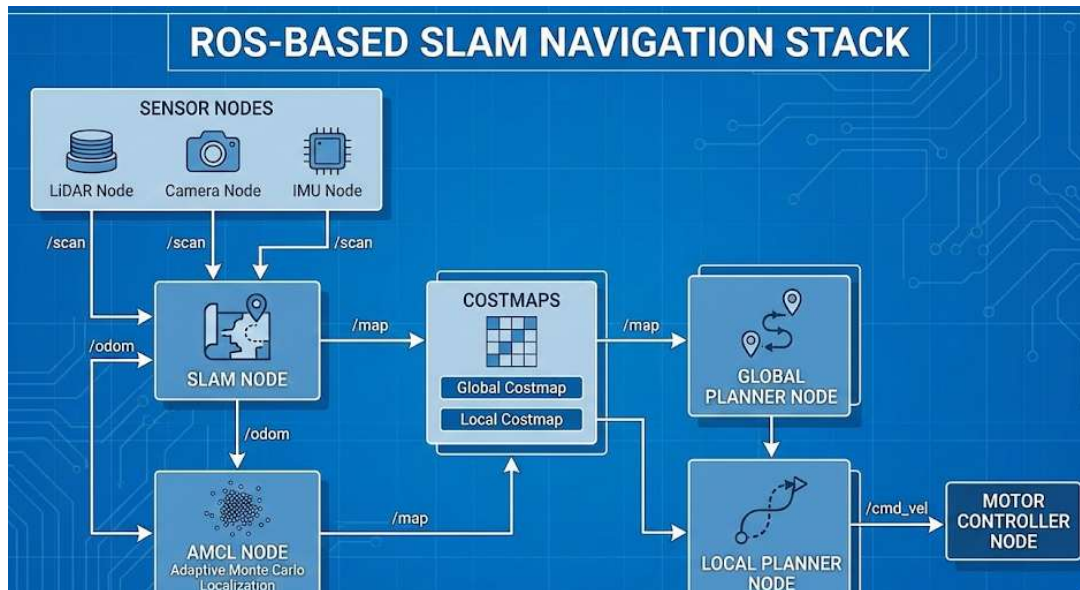


Figure: ROS-based SLAM navigation stack: sensor nodes, SLAM node, AMCL, costmaps, global and local planner nodes.

- **Global Planner:** A node implementing global planning (e.g. navfn or global_planner). Given the current robot pose (from AMCL) and a goal, it computes a global path through the static map. This node usually only updates when goals change or significant map updates occur.
- **Local Planner (DWA):** The move_base stack's local planner (e.g. dwa_local_planner) generates velocity commands by considering the robot's dynamic constraints and the local costmap. It continuously replans on a short horizon to avoid obstacles, using the latest sensor data.
- **Costmaps:** Two layered costmaps – a global costmap for the whole floor, and a local costmap around the robot. The global costmap is usually just the static map inflated by the robot's radius. The local costmap is updated from the robot's sensors at runtime, including detected dynamic obstacles. Both costmaps should be tuned carefully: see below.
- **Recovery Behaviors and Monitoring:** If the robot gets stuck (local planner fails), the system can trigger recovery behaviors (clear local costmap, rotate robot, re-plan). A supervisor node monitors health (e.g. stuck in elevator).

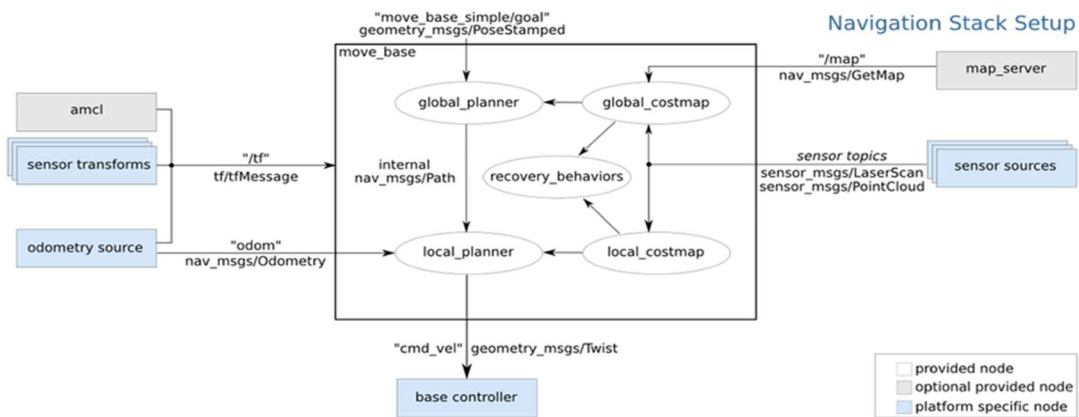


Figure: Example ROS navigation stack combining SLAM (for mapping and localization) with layered planning.

Costmap Tuning and Dynamic Obstacles

Costmap Tuning: The costmaps must be tuned for the robot size and environment:

- **Resolution:** Choose a resolution fine enough to detect obstacles but coarse enough to be computationally efficient (e.g. 5–10 cm per cell for indoor corridors). Finer grids give more accurate obstacle shapes but increase memory and CPU use.
- **Inflation Radius:** Inflate obstacles by the robot's radius plus a safety margin (e.g. if robot has 0.3 m radius, set inflation ~0.4–0.5 m). Inflation creates a "buffer zone" so the robot does not pass too close to obstacles. In tight academic corridors, high inflation can block narrow passages, so balance is needed.
- **Obstacle Lifecycle:** Obstacles (especially dynamic ones) should time out if not sensed again. The local costmap should be cleared of obstacles not visible after some time (`observation_persistence`). Similarly, clearable regions (e.g. previously occupied cells) must be updated by fresh sensor data.
- **Sensor Integration:** The costmap parameters must match the sensor properties. For example, a tilted LiDAR might detect low obstacles; the `max_obstacle_height` parameter should cover the highest relevant obstacle (e.g. 2 m). Layers of sensors (2D LiDAR, possibly RGB-D) can be fused.
- **Footprint:** Define the robot's footprint polygon accurately. An incorrect footprint can cause either collisions (if footprint is undersized) or overly conservative paths (if oversize).

Obstacle Inflation: Inflation uses a cost function that decays with distance from an obstacle, typically quadratic. This ensures that even after passing an obstacle, the costmap reflects the clearance. A key parameter is `cost_scaling_factor` controlling how steeply cost decays. In narrow corridors, a larger inflation may treat much of the corridor as high-cost; tune so the robot still finds a path.

Dynamic Obstacles (Humans): Handling moving people requires: - Up-to-date sensor data: The local planner uses the latest laser scans (or depth camera data) to detect people as obstacles in the local costmap. - Costmap Clearing: Ensure the local costmap is cleared and updated quickly, so moving obstacles do not leave “ghost obstacles.” Parameters like `clearing_rotation_trajectory` can help, or the robot should be able to backtrack if needed. - Predictive Planning: While DWA replans at ~10 Hz, it may not predict pedestrian motion. In practice, if a human blocks the path, the robot stops or slowly navigates around. One can implement a simple sensor-based brake if an obstacle is too close. - Behavior: For safety, limit robot speed near people (tunable `max_vel_x/max_rot`). Also, recovery behaviors can re-plan if stuck behind a crowd.

Multi-Floor Specifics: When planning a route that involves changing floors, the planner needs to incorporate elevator/stair information. One approach is hierarchical: plan a path on the current floor to the elevator, then switch to the floor above’s map and plan to the destination. Alternatively, embed elevator locations in the map as special navigable areas (with transitional cost). In a fully unified multi-floor map, global planning would operate in 3D (but most ROS stacks are 2D). In practice, the deployment might treat each floor separately and handle transitions as discrete events

5. Critical Evaluation:

Design choices involve trade-offs:

- **CPU Load vs. Accuracy:** High-accuracy SLAM demands more computation. On a limited CPU, one may choose simpler SLAM (Gmapping or Hector) and accept some drift, or run complex SLAM at lower frame rates. Similarly, using many AMCL particles improves localization accuracy at the cost of CPU. One mitigation is to offload heavy computation to specialized hardware or to reduce map resolution / search windows where possible.

- **Drift vs. Loop Closure Frequency:** Aggressive loop closure corrects drift but can be CPU-intensive (solving the large pose-graph). If the robot rarely revisits the same places, loop constraints are sparse and drift accumulates. Weigh the benefit of frequent loop detection (and slower cycle) against light, incremental mapping. For example, Cartographer continuously optimizes submaps, whereas Gmapping only occasionally re-samples the best particle for a loop.
- **Sensor Types: LIDAR vs Camera vs IMU vs Odometry:** Each has pros and cons. LiDAR provides accurate distance but fails on transparent/reflective surfaces. Cameras see textures and colors, but fail in darkness and with motion blur. IMUs provide high-rate motion data but drift. Fusing them gives robustness: e.g. a LiDAR-IMU SLAM (like LIO-SAM) can detect altitude changes via IMU during elevator rides, and use LiDAR to build planar maps. If Wi-Fi or external positioning (e.g. Bluetooth beacons) were available, they could assist, but in this scenario assume self-contained sensors only.

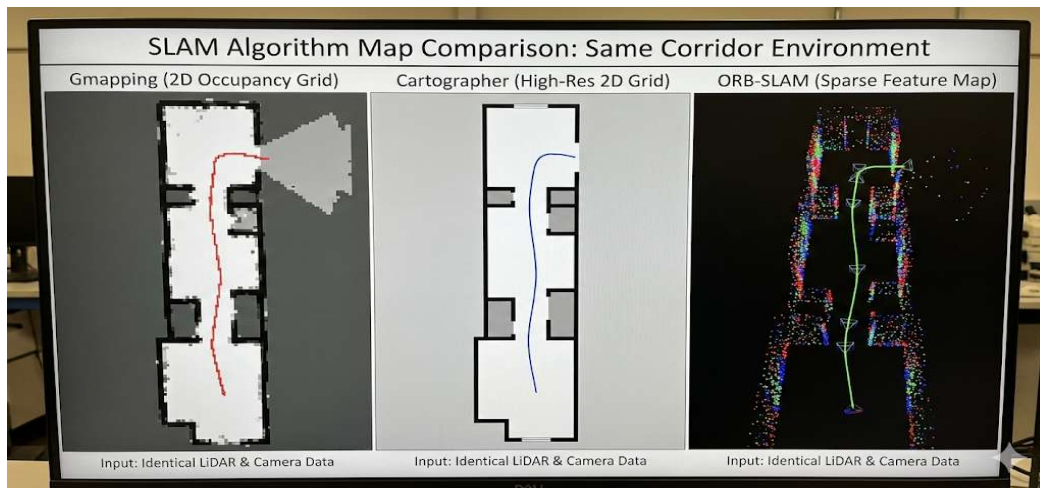


Figure: Side-by-side image comparison: Gmapping vs Cartographer vs ORB-SLAM maps in the same corridor.

Failure Modes: Common failure modes include:

- **Lost Pose:** In featureless areas, the robot may lose track. Recovery by re-localization (e.g. reinitializing the filter).
- **Cycle Errors:** Turning an elevator or ascending stairs without correct altitude update may cause mapping inconsistencies (mistaking the height change as horizontal drift). Solution: use IMU or barometer to detect vertical motion and insert the correct transform.

- **Loop Mis-association:** Thinking two similar-looking corridors are the same place (especially if the map is being built on-the-fly). One must then either manually intervene or design the loop closure to be conservative (e.g. require strong feature match).
- **Obstacle Dynamics:** A moving obstacle might be incorporated temporarily into the map (static map error). Mitigation: update map only after verifying consistency (Cartographer's submap approach helps, and one can filter out dynamic objects by time).
- **Communication Loss:** If running distributed SLAM or map sharing via Wi-Fi, dropout can freeze localization. The system should be robust by default (e.g. not relying on remote voxels).

Mitigation Strategies:

- **Sensor Fusion:** Combine LiDAR and vision so that the failure of one is compensated by the other.
- **Environmental Aids:** If allowed, place artificial features (markers, AprilTags) in problematic areas (e.g. at the top of a stairwell) to aid relocalization.
- **Map Segmentation:** Build and store separate maps per floor; switch maps based on reliable indicators (floor sensors) to avoid cross-floor confusion.
- **Adaptive Algorithms:** For example, increase AMCL particle spread in ambiguous regions, or use odometry corrections when GPS/Wi-Fi is lost.
- **Resource Constraints:** Given limited onboard computation, one might disable expensive features when unnecessary. For instance, run SLAM at a slower update rate when stationary, or reduce LiDAR scan rate in open areas. The architecture can also allow asynchronous updates: the SLAM node can lag slightly behind real-time if needed, as long as the local planner has most recent laser scans directly.

6. Applications:

The discussed SLAM and navigation strategies directly map to real-world indoor mobile robots:

- **Campus Delivery Robots:** Universities deploy robots to deliver mail, packages, or food between buildings and floors. These robots encounter long hallways, elevators, and busy students. A suitable design uses a 2D LiDAR-based SLAM (to map each building floor) and AMCL for localization, with predefined elevator stops. Global planning integrates

floor connections, and the robot must slow or stop for people. For example, a campus courier robot may build a layered map (3rd floor of Hall A, 2nd floor of Hall B, etc.) and plan routes that include an elevator as an edge in the graph.

- **Cleaning Robots (Janitorial):** These roam office buildings or dormitories. They use SLAM to cover all corridors and rooms. The challenges include dynamic obstacles (people, moving chairs) and glass walls (which can trap robot). Often, the cleaning area is known, so these robots build a static map and then repeatedly localize and plan within it. They benefit from a robust local planner (to avoid bumping into people) and can sometimes rely on prior floor plans to speed mapping.
- **Warehouse and Inventory Robots:** In multi-level storage (e.g. parts warehouses with mezzanines), robots use SLAM to navigate between aisles and floors via lifts or ramps. The environment is structured but repetitive. Here, high accuracy is crucial to reach the right shelf. These robots may use visual fiducials (QR codes) to mark transition points (elevator doors). They often run powerful SLAM (sometimes with 3D LiDAR) and have centralized monitoring, but our strategies apply.
- **Service Robots (Hospitals, Malls):** Hospital delivery robots face similar corridors and crowd issues. They often have to navigate on schedules (time-varying map occupancy). Our design of costmap clearing and recovery is essential here. Similarly, mall or office guide robots deal with Wi-Fi dead zones (e.g. basement) and must be fully autonomous.

In all these cases, the system designer leverages SLAM to build initial maps (often during commissioning of the robot) and then uses those maps with robust localization for daily tasks. The key insight is matching the SLAM and planning approach to the environment: e.g. highly-structured warehouses might allow simpler mapping, whereas complex open-plan buildings might require richer SLAM.

7. Conclusion and Future Scope:

Robust SLAM-based navigation in multi-floor indoor environments requires balancing algorithmic strengths against environmental challenges and hardware limits. Our analysis suggests a hybrid approach: use a LiDAR-based SLAM for primary mapping (due to its loop closure and multi-trajectory support), complemented by camera-based localization when needed for additional landmarks. Localization can rely on AMCL

with map switching per floor. A global planner ensures long-range route planning (incorporating floor transitions), while a local planner (DWA) handles real-time obstacle avoidance. Careful costmap tuning and recovery strategies mitigate issues like narrow corridors and dynamic humans.

For future work, several directions are promising. Multi-robot mapping could allow different robots to map separate floors concurrently and merge their maps for a complete building model. The emerging ROS 2 Navigation2 stack offers improved real-time performance and lifecycle management, which could be beneficial for on-the-fly map updates and multi-floor coordination. Visual-Inertial Odometry (VIO) algorithms like ORB-SLAM3 with IMU or VINS-Mono are maturing; integrating VIO can help in low-feature areas where LiDAR struggles (e.g. glass atria) by providing alternate motion cues. Additionally, leveraging semantic information could enhance loop closure by giving unique context to otherwise repetitive corridors. Finally, GPU acceleration or edge AI chips could allow more complex SLAM (deep learning-based place recognition or semantic mapping) to run within the limited onboard compute budget. These advancements will push multi-floor SLAM toward even greater autonomy and reliability in complex indoor settings.